



**ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA**
Universidad de Córdoba



**PRÁCTICA 2 – Código limpio y
refactorización**
Grado en Ingeniería Informática

Raúl Sánchez Flores

Curso 2025 / 2026

GitHub: <https://github.com/raulsf01/MDAS>



UNIVERSIDAD DE CÓRDOBA



Índice:

1:	<i>Semana 1: Reglas de nombrado.....</i>	<i>Pág. [2]</i>
2:	<i>Semana 2: Reglas de comentarios y formato.....</i>	<i>Pág. [4]</i>
3:	<i>Semana 3 y 4: Reglas de funciones y refactorización manual.....</i>	<i>Pág. [6]</i>
4:	<i>Semana 5: Refactorización automática.....</i>	<i>Pág. [8]</i>

Semana 1: Reglas de nombrado

En gestorClinica.java nombre de función:

```
// Refactor Semana 1: usar letra Mayúscula al inicio de declaracion.
//void MostrarTodasLasMascotas() {
void mostrarTodasLasMascotas() {
    for (Map.Entry<String, Mascota> e : mascotas.entrySet()) {
        System.out.println(e.getKey() + " -> " + e.getValue());
    }
}
```

En repositorio.java variables sin significado:

```
// Refactor Semana 1: regla 6 y 8, unica letra o muy corta
//public T buscarPorIndice(int i){
public T buscarPorIndice(int index){
    if(index < 0 || index >= elementos.size()){//si el indice
        return null;
    }else{// si no devuelve el indice
        return elementos.get(index);
    }
}
```

En propietario.java variables sin significado:

```
// Refactor Semana 1: usar palabra confusa para refereirse a un objeto.
/*public Propietario(Propietario otro) { //copia profunda
    this.DNI = otro.DNI;
    this.nombre = otro.nombre;
    this.telefono = otro.telefono;
}*/
public Propietario(Propietario propietarioOriginal) { //copia profunda
    this.DNI = propietarioOriginal.DNI;
    this.nombre = propietarioOriginal.nombre;
    this.telefono = propietarioOriginal.telefono;
}
```

```
//Refactor semana 1: variables sin significado
/*public boolean equals(Object o) {
    if (!(o instanceof Propietario that)) return false;
    return Objects.equals(DNI, that.DNI);
}*/
public boolean equals(Object propietario) {
    if (!(propietario instanceof Propietario that)) return false;
    return Objects.equals(a: DNI, b: that.DNI);
}
```

En veterinario.java nombre de variable mal declarada:

```
// Refactor semana 1: nombre de variable extraño
//private String especalidad;
private String especialidad;
```

En main.java uso de letras para referirse a objetos:

```
// Refactor Semana 1: usar i para refereirse a un empleado.
//for ( Empleado i : empleados){
for (Empleado currentWorker : empleados) {
    System.out.println(x: currentWorker.toString());
}
```

Estas dos siguientes se repiten por todo el main. No se añaden más capturas por no ser repetitivo pero se puede comprobar su arreglo en el archivo.

```
// Refactor semana 1: variables repetitivas sin significado
System.out.println("p1-> "+ p1);
System.out.println("p2-> "+ p2);
```

```
// Refactor semana 1: variables repetitivas sin significado
System.out.println(x: m1);
System.out.println(x: m2);
System.out.println(x: m3);
```

```
// Refactor semana 1: de nuevo nombres sin significado
Empleado[] empleados = {v1,a1,a2}; //metemos a una array cosas ya creadas
```

Semana 2: Reglas de comentarios y formato

En gestorClinica.java los comentarios son para explicar cosas obvias sobre el funcionamiento que cualquier programador debería conocer. (Los eliminamos)

```
public class GestorClinica {
    private Map<String, Mascota> mascotas;
    private Set<String> especies;

    public GestorClinica() {
        this.mascotas = new HashMap<>(); //hacemos que mascotas sea un HashMap
        this.especies = new HashSet<>();
    }

    public void registrarMascotas(Mascota m) {
        mascotas.put(key:m.getNombre(), value: m); //añadimos el nombre de la mascota Q YA ESTA CREADA
        especies.add(e: m.getEspecie());
    }

    public Mascota buscarPorNombre(String nombre) {
        return mascotas.get(key:nombre); //busca el nombre si esta devuelve el nombre, si no pues es null
    }

    void mostrarEspecies() {
        System.out.println("Especies registradas: " + especies);
        // como el HashSet NO permite repeticiones pues que se imprima todo y ya
    }
}
```

En main.java comentarios de prueba sobre algo similar a control de versiones/funcionamiento para comprobar que al introducir datos negativos no se guardan, y apuntes para declarar arrays:

```
mascota3.setEdad(edad:-3); //intentar cambiar la edad a negativa
System.out.println("Intentamos cambiar la edad a -3-> "+mascota3); //vemos q no cambia

System.out.println("Total mascotas-> "+Mascota.getTotalMascotas());

//EJERCICIO 3
System.out.println();
System.out.println(x: "EJERCICIO 3");

* //Empleado[] empleados = new Empleado[3]; //creamos array
Empleado veterinario1 = new Veterinario(nombre: "Pepito", salarioBase:1800, especialidad:"Cirujano");
Empleado auxiliar1 = new Auxiliar(nombre: "Josefina", salarioBase:1400, turno: "mañana");
Empleado auxiliar2 = new Auxiliar(nombre: "Lucia", salarioBase:1400, turno: "noche");
```

Apuntes innecesarios sobre funcionamiento:

```
List<Informable> informables = new ArrayList<>();
informables.add((Informable) veterinario1); //pone eso pq informable esta en veterinario y yo he creado emp,eado = veterinario no veterinario=veterinario
informables.add(e: mascota1);
informables.add(e: mascota2);
```

De nuevo,

```
propietario:propietario4)); // especie repetida
```

Y otra caso sobre funcionamiento básico en mascota.java, este archivo está lleno de comentarios de este estilo haciendo aclaraciones obvias:

```
this.edad = edad;
this.propietario = propietario;
totalMascotas++; //asi cada vez que se genere una mascota se suma 1

private propietario propietario,
private static int totalMascotas=0; //atributo privado estatico

public static int getTotalMascotas() { // Metodo estático getTotalMascotas() que devuelva el valor del contador.
    return totalMascotas;
}

public void setEdad(int edad) {
    if (edad>=0){
        this.edad = edad;
    }
}

public boolean esCachorro() { //Método esCachorro() que devuelva true si la edad es menor de 2.
    return edad<2;
}
```

En propietario.java pasa exactamente igual, aunque solo con un método:

```
public void setTelefono(String telefono) { //solo asigna valor si la cadena tiene 9
    if (telefono.length()==9){
        this.telefono = telefono;
    }
}
```

Y para terminar, en repositorio.java igual, aclaraciones simples y explicación de cómo funciona una simple comparación:

```
public int totalElementos() { //numero de elementos
    return elementos.size();
}

public T buscarPorIndice(int index){
    if(index < 0 || index >= elementos.size()) { //si el indice es negativo Y es mayor a la cantidad de elemento q hay es null
        return null;
    } else { // si no devuelve el indice
        return elementos.get(index);
    }
}
```

Semana 3 y 4: Reglas de funciones y Refactorización manual

En cuanto a los archivos de clases, están todos realizados con funciones sencillas, la mayoría, los famosos “getters y setters”, añadir y eliminar.

Estas no necesitan mejora, así que nos podemos centrar en el main, que es un verdadero caos.

El main consta de 200 líneas aproximadamente en las que el propietario del proyecto ponía a prueba el mismo, pero en este están todos los datos mezclados y no hay un orden como tal. Además no se puede elegir qué probar, osea que si se desea realizar la prueba de un método, hay que buscar dentro de un output gigantesco dónde se imprime la línea concreta y ver si falla o no.

Como solución y refactorización manual, se crean nuevos métodos secundarios para hacer las pruebas y se hace un menú en el main en el que poder elegir qué comprobar.

Pasamos de líneas y líneas así:

```
public class Main {
    public static void main(String[] args) { // Refactor Semana 1: metodo no estaba declarado publico, por tanto, ilegible para otro compilador al exportar

        System.out.println();
        System.out.println(e: "EJERCICIO 1");
        Propietario propietario1 = new Propietario(dni: "12345678", nombre: "Ana Garcia", telefono: "612345678");
        Propietario propietario2 = new Propietario(propietarioOriginal: propietario1);
        propietario2.setTelefono(telefono: "999888777");

        // Refactor semana 1: variables repetitivas sin significado
        System.out.println("propietario1-> " + propietario1);
        System.out.println("propietario2-> " + propietario2);

        //EJERCICIO 2 MASCOTAS
        System.out.println();
        System.out.println(e: "EJERCICIO 2");
        Mascota mascota1 = new Mascota(nombre: "Rex", especie: "Perro", edad: 3, propietario: propietario1);
        Mascota mascota2 = new Mascota(nombre: "Lucky", especie: "Perro", edad: 12, propietario: propietario2);
        Mascota mascota3 = new Mascota(nombre: "Felix", especie: "gato", edad: 1, propietario: propietario1);

        // Refactor semana 1: variables repetitivas sin significado
        System.out.println(e: mascota1);
        System.out.println(e: mascota2);
        System.out.println(e: mascota3);

        System.out.println("Total mascotas-> " + Mascota.getTotalMascotas());

        //EJERCICIO 3
        System.out.println();
        System.out.println(e: "EJERCICIO 3");

        Empleado veterinario1 = new Veterinario(nombre: "Pepito", salarioBase: 1800, especialidad: "Cirujano");
        Empleado auxiliar1 = new Auxiliar(nombre: "Josefina", salarioBase: 1400, turno: "Mañana");
        Empleado auxiliar2 = new Auxiliar(nombre: "Lucia", salarioBase: 1400, turno: "Noche");

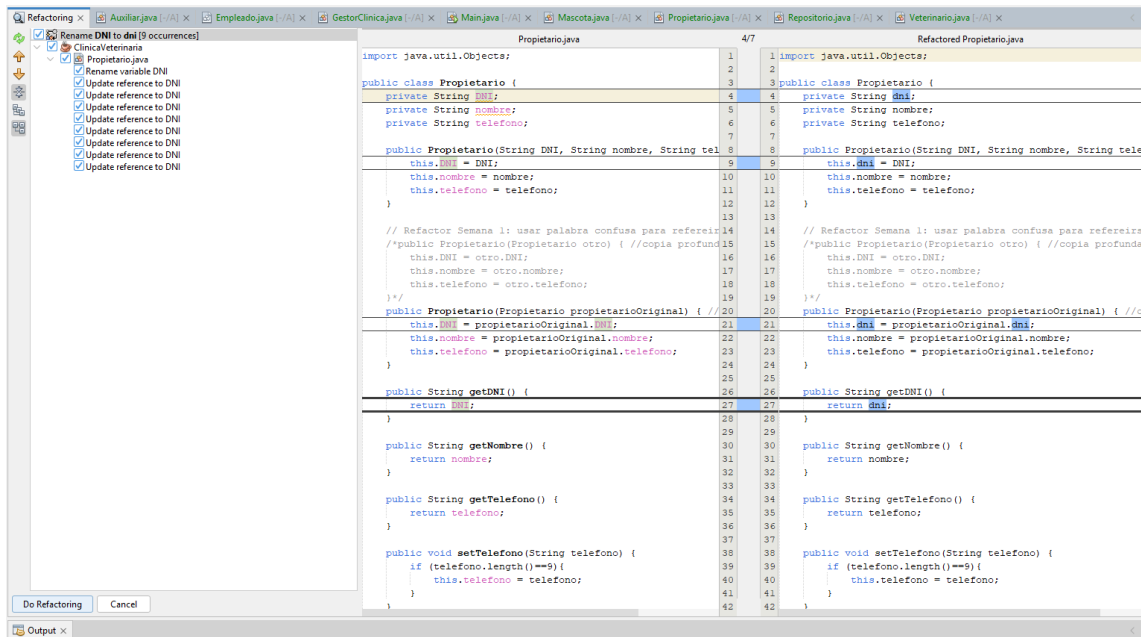
        // Refactor semana 1: de nuevo nombres sin significado
        Empleado[] empleados = {veterinario1, auxiliar1, auxiliar2}; //metemos a una array cosas ya creadas
```

A un menú principal organizado y funciones separadas:

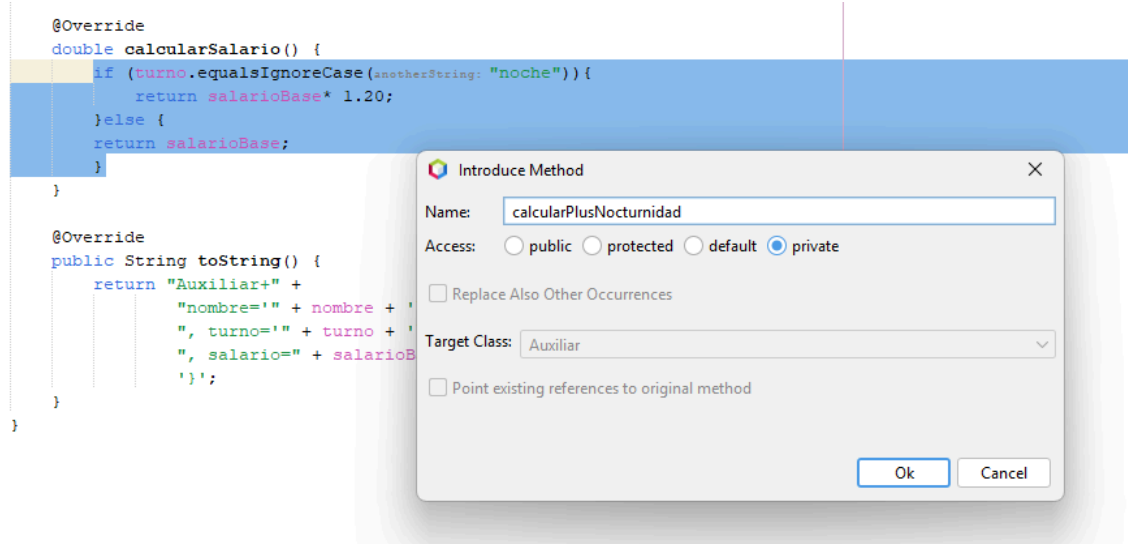
```
public class Main {  
  
    public static void main(String[] args) {  
        Scanner teclado = new Scanner(System.in);  
        int opcionSeleccionada;  
  
        do {  
            System.out.println("\n=====");  
            System.out.println(" SISTEMA DE GESTIÓN CLÍNICA VETERINARIA ");  
            System.out.println("=====");  
            System.out.println("1. Pruebas de Registro de Propietarios");  
            System.out.println("2. Pruebas de Control de Mascotas");  
            System.out.println("3. Pruebas de Contratos de Empleados y Salarios");  
            System.out.println("4. Pruebas de Generación de Informes Polimórficos");  
            System.out.println("5. Pruebas de Repositorio Genérico");  
            System.out.println("6. Pruebas de Búsqueda en Gestor de Clínica");  
            System.out.println("7. Pruebas de Comparación Genérica de Objetos");  
            System.out.println("8. Pruebas de Procesamiento con Streams y Comparadores");  
            System.out.println("0. Salir de la aplicación");  
            System.out.print(": Seleccione una opción: ");  
  
            try {  
                opcionSeleccionada = teclado.nextInt();  
            } catch (InputMismatchException e) {  
                System.out.println("Error: Por favor, introduzca un número válido.");  
                teclado.next();  
                opcionSeleccionada = -999;  
                continue;  
            }  
  
            switch (opcionSeleccionada) {  
                case 1 -> probarRegistroPropietarios();  
                case 2 -> probarControlMascotas();  
                case 3 -> probarContratosYSalariosEmpleados();  
                case 4 -> probarGeneracionInformesPolimorficos();  
                case 5 -> probarAlmacenamientoRepositorioGenerico();  
                case 6 -> probarBusquedasGestorClinica();  
                case 7 -> probarComparacionGenericaMayor();  
                case 8 -> probarFiltradoYEstadisticasConStreams();  
                case 0 -> System.out.println("Saliendo del programa de pruebas...");  
            }  
        }  
    }  
}
```

Semana 5: Refactorización automática

Cambiamos DNI por dni:



Ejemplo de inserción de métodos automática:



```
@Override
double calcularSalario() {
    return calcularPlusNocturnidad();
}

private double calcularPlusNocturnidad() {
    if (turno.equalsIgnoreCase(anotherString: "noche")) {
        return salarioBase* 1.20;
    } else {
        return salarioBase;
    }
}
```

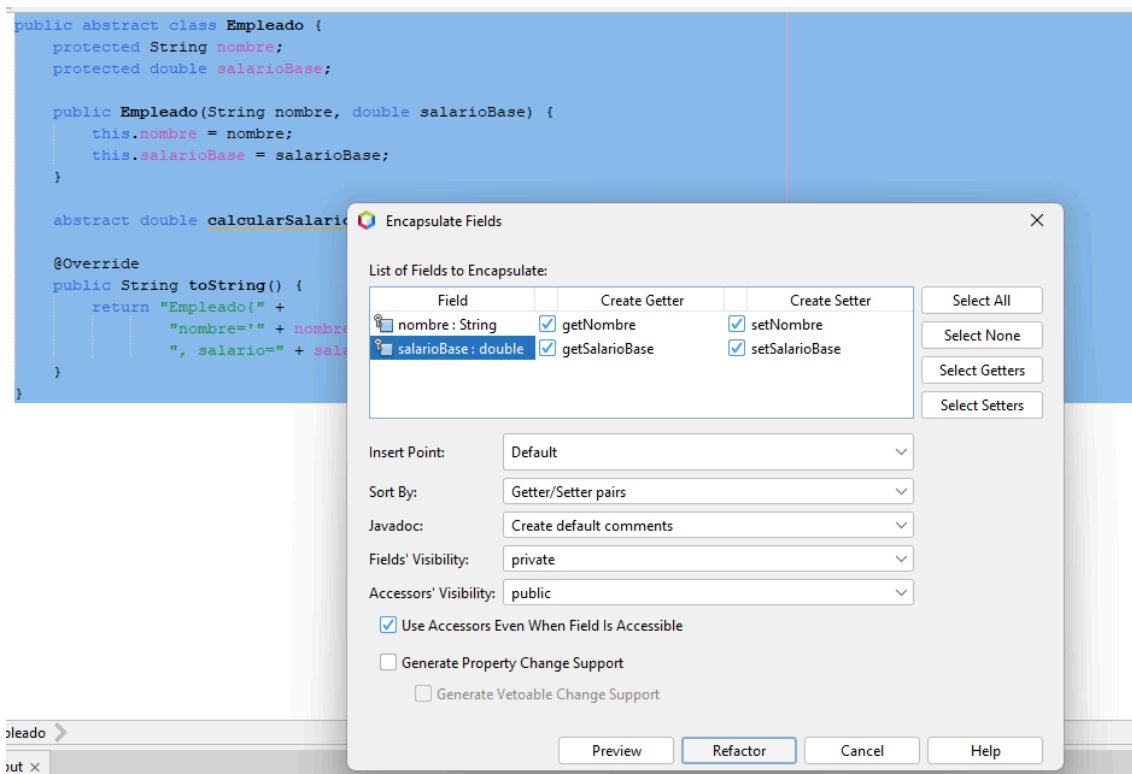
En empleado.java hacemos un refactor para eliminar los atributos protected y añadir los comunes get y set a la clase y pasamos de esto:

```
public abstract class Empleado {
    protected String nombre;
    protected double salarioBase;

    public Empleado(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    abstract double calcularSalario();

    @Override
    public String toString() {
        return "Empleado{" +
            "nombre='" + nombre + '\'' +
            ", salario=" + salarioBase + '€';
    }
}
```



A una clase más profesional:

```
public abstract class Empleado {  
  
    /**...3 lines */  
    public String getNombre() {  
        return nombre;  
    }  
  
    /**...3 lines */  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
  
    /**...3 lines */  
    public double getSalarioBase() {  
        return salarioBase;  
    }  
  
    /**...3 lines */  
    public void setSalarioBase(double salarioBase) {  
        this.salarioBase = salarioBase;  
    }  
    private String nombre;  
    private double salarioBase;  
  
    public Empleado(String nombre, double salarioBase) {  
        this.nombre = nombre;  
        this.salarioBase = salarioBase;  
    }  
  
    abstract double calcularSalario();  
  
    @Override  
    public String toString() {
```